Modern Cloud Computing

Lecture 17: Heterogeneous Computing in the Cloud II

Chongzhou Fang

Rochester Institute of Technology

Backgrounds

Heterogeneous Resource Serving in Cloud

- Heterogeneous resources: CPUs, GPUs of many generations and vendors, NICs, ...
- Challenges:
  - Resource fragmentation: stranded GPUs, CPU:GPU mismatch, topology constraints...
  - Underutilization: reserved-but-idle capacity, low GPU SM utilization...
  - Hardware-aware software optimization needed
  - ...

RDMA (Remote Direct Memory Access)

Background: RDMA

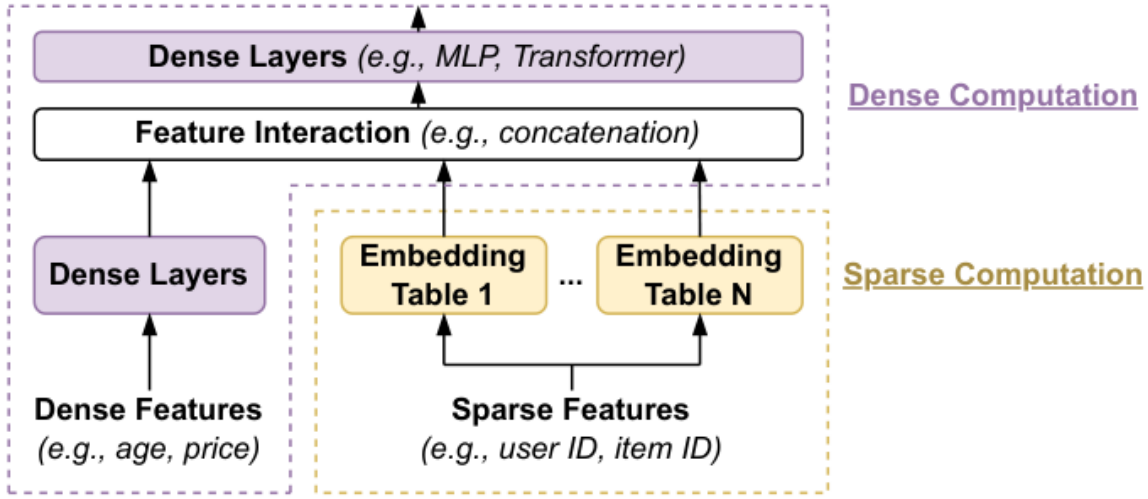
- **RDMA:** Direct memory access from the memory of one computer into that of another without involving either computer's OS.
- **RNIC:** An RDMA-capable network adapter.
- **GPUDirect RDMA:** Enables a direct path for data exchange between the GPU and a third-party peer device using standard features of PCIe.

(Sources: NVIDIA, *RDMA Aware Networks Programming Guide*, Key Concepts; NVIDIA, *GPUDirect RDMA* documentation;)

Part I: GPU-Disaggregated Serving for Deep Learning Recommendation Models at Scale

Deep Learning Recommendation Models (DLRMs)

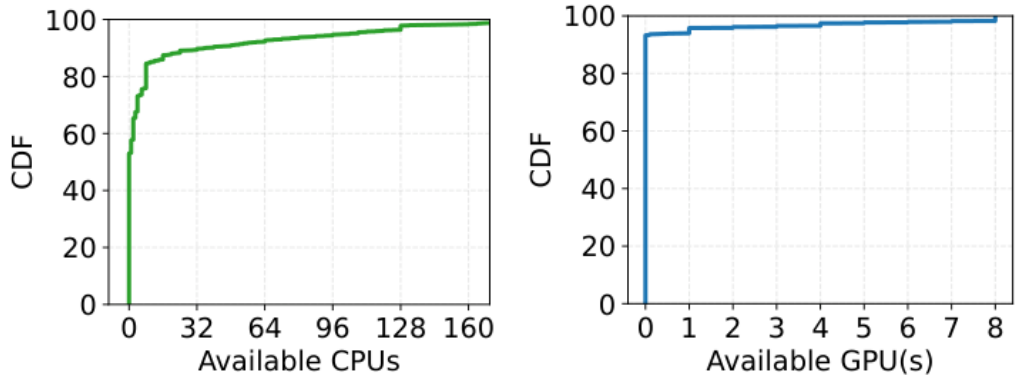
- >79% of Meta's AI inference cycles
- Huge **embedding tables** (100 GB to 1 TB): memory-bound, low arithmetic intensity, best on many CPU cores
- MLP / transformer layers on the gathered embeddings: 10-44x faster on a GPU
- A typical instance: **48 CPUs + 1 GPU**, with a tens-of-ms latency SLO



(Source: Yang et al., "GPU-Disaggregated Serving for DLRMs at Scale," NSDI 2025, Fig. 2)

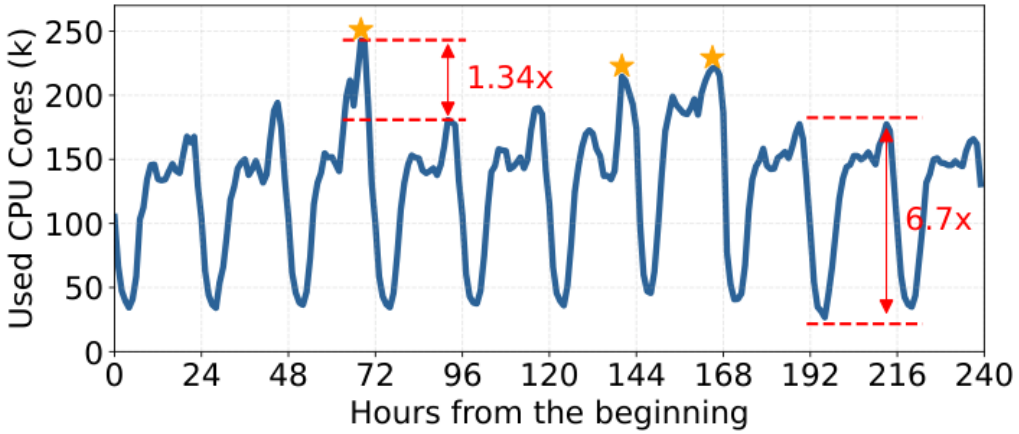
Two Provisioning Problems at Alibaba

C1: Fragmentation in daily serving



- High allocation rates
- Hard to scale due to severe fragmentation
  - DLRM instances: High CPU-to-GPU ratio

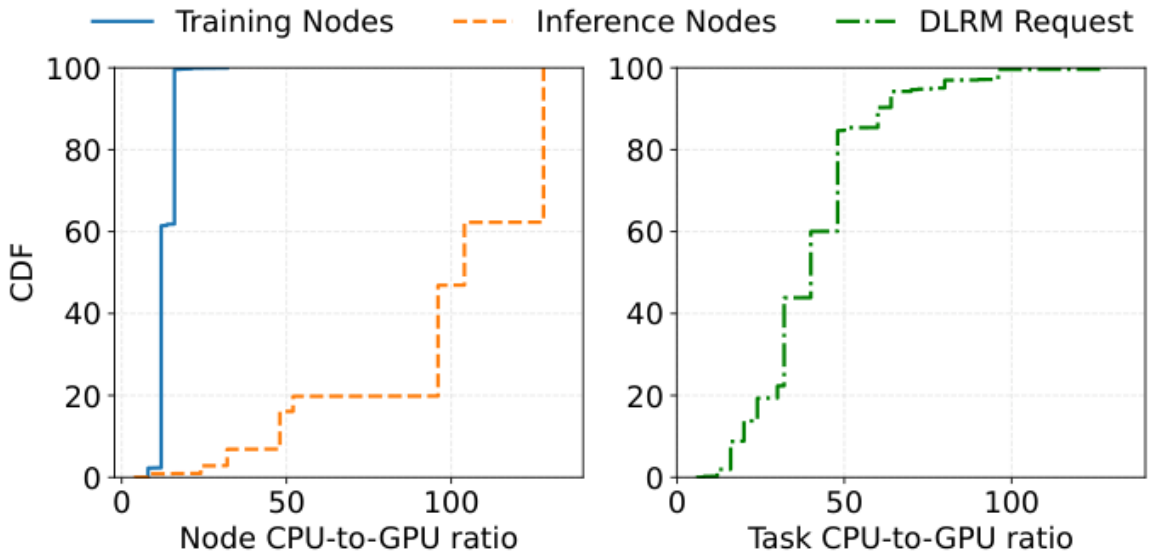
C2: Spikes and failed capacity loans



- Over-provisioning: underutilization
- Mismatch between server config and resource demand renders capacity loaning ineffective

(Source: Yang et al., NSDI 2025, Figs. 1 and 3)

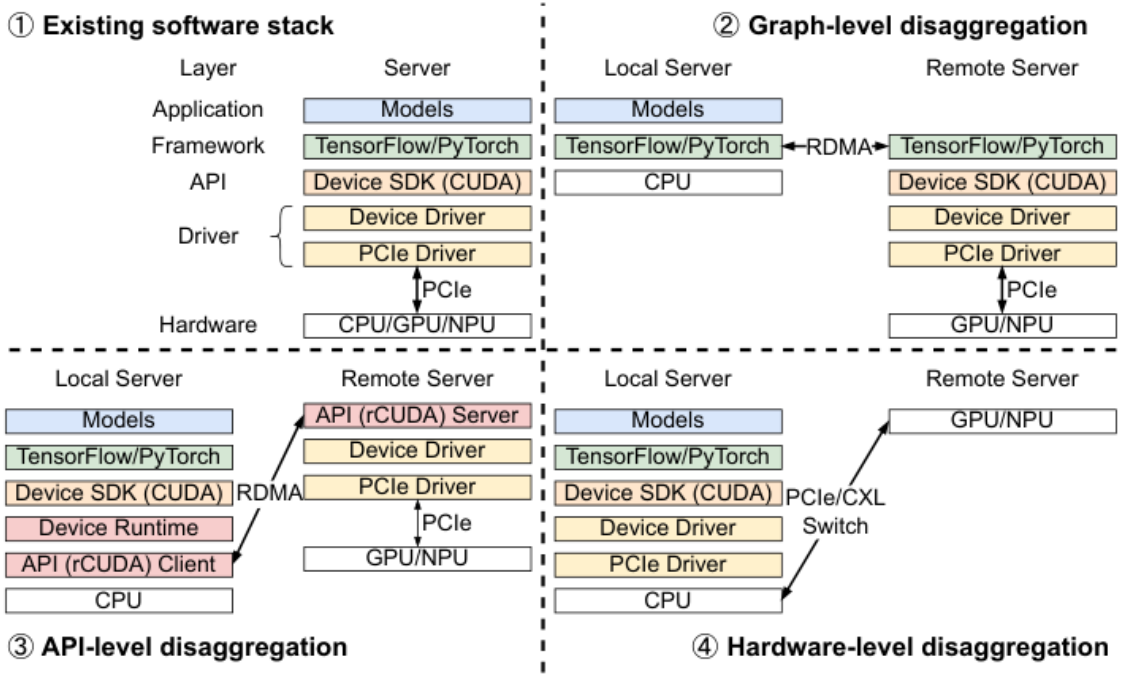
The Mismatch: Server Shape vs. Workload Shape



(Source: Yang et al., NSDI 2025, Fig. 4)

- Training: 8 x GPU, 16-32 cores, **CPU:GPU < 20**
- DLRM: 1 GPU, ~128 cores, ratio ~100
- **>90% of DLRMs need a ratio above 20**
- Example: A 96-core, 8-GPU training server fits **two** DLRM instances; the CPUs are gone and **6 GPUs are stranded**

Where to Disaggregate the GPU?

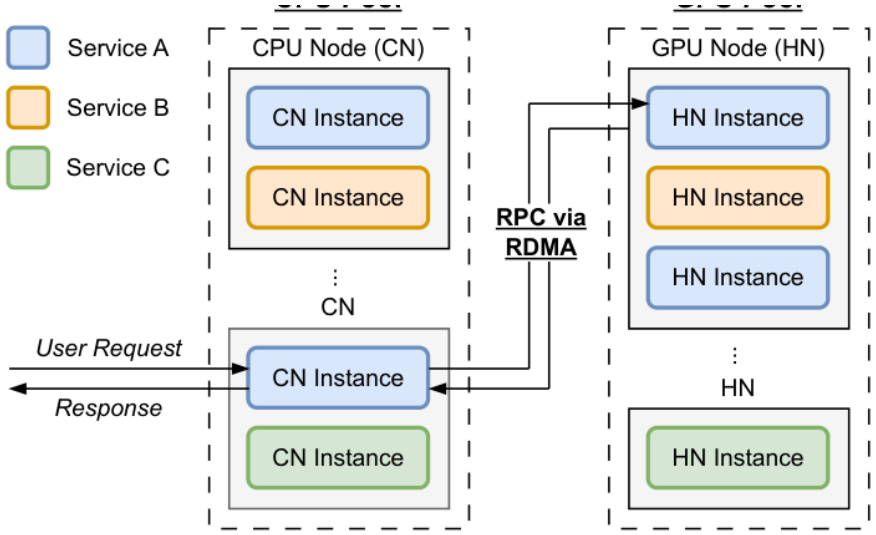


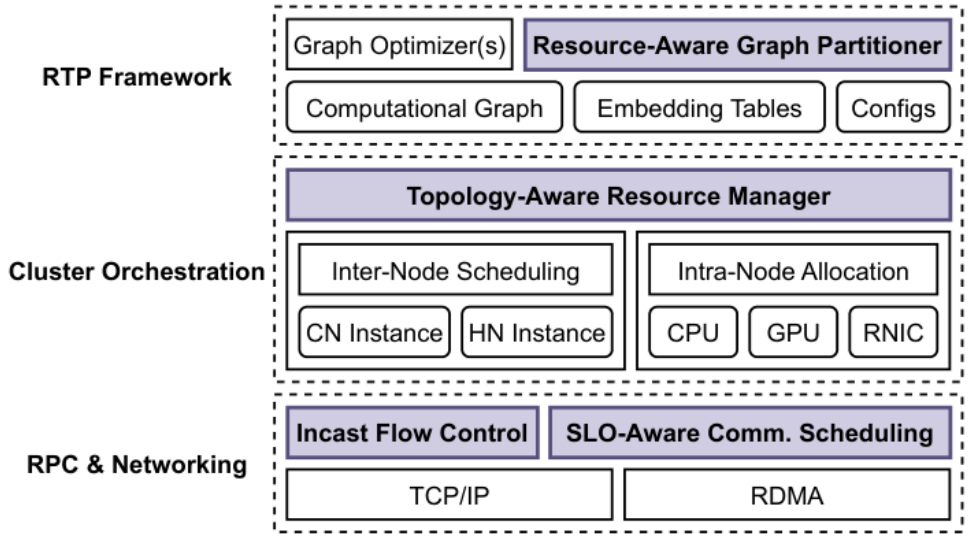
(Source: Yang et al., NSDI 2025, Fig. 5)

Enablers: Fast RDMA networks (200-400 Gbps RNICs rival PCIe 4.0 x16 (256 Gbps))

- **API level** (rCUDA, Bitfusion): intercept CUDA calls and forward them to a remote GPU; framework-agnostic but little room to optimize, chases every CUDA release
- **Hardware level** (multi-hop PCIe switches, CXL 3.0): transparent and fast, but needs new hardware and stays within a rack
- **Graph level** (Prism): split the model's compute graph into a CPU subgraph and a GPU subgraph, run them on different servers over RDMA (Remote Direct Memory Access)

Prism Overview

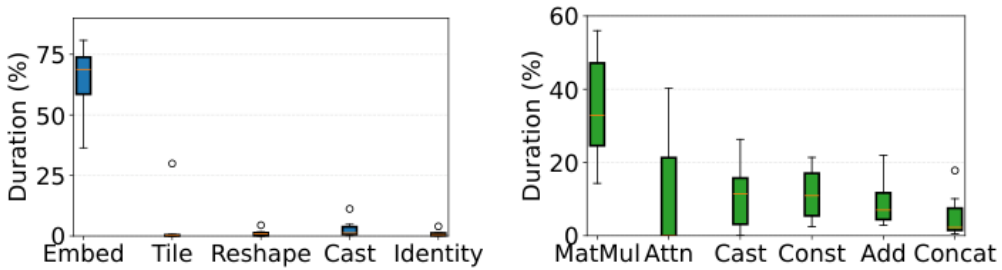




- **CPU nodes (CNs):** many cores, big memory, no GPU. **Heterogeneous GPU nodes (HNs):** multi-GPU training servers with modest CPU. HNs and CNs connect via high-speed RDMA network
- Request -> CN instance (embedding lookup, pre/post-processing) -> RDMA RPC -> HN instance (dense compute) -> back
- Deployed since 2022 on **>10k GPUs, 20k instances, 800k CPUs**

(Source: Yang et al., NSDI 2025, Figs. 6-7)

Key Component 1: Resource-Aware Graph Partitioner

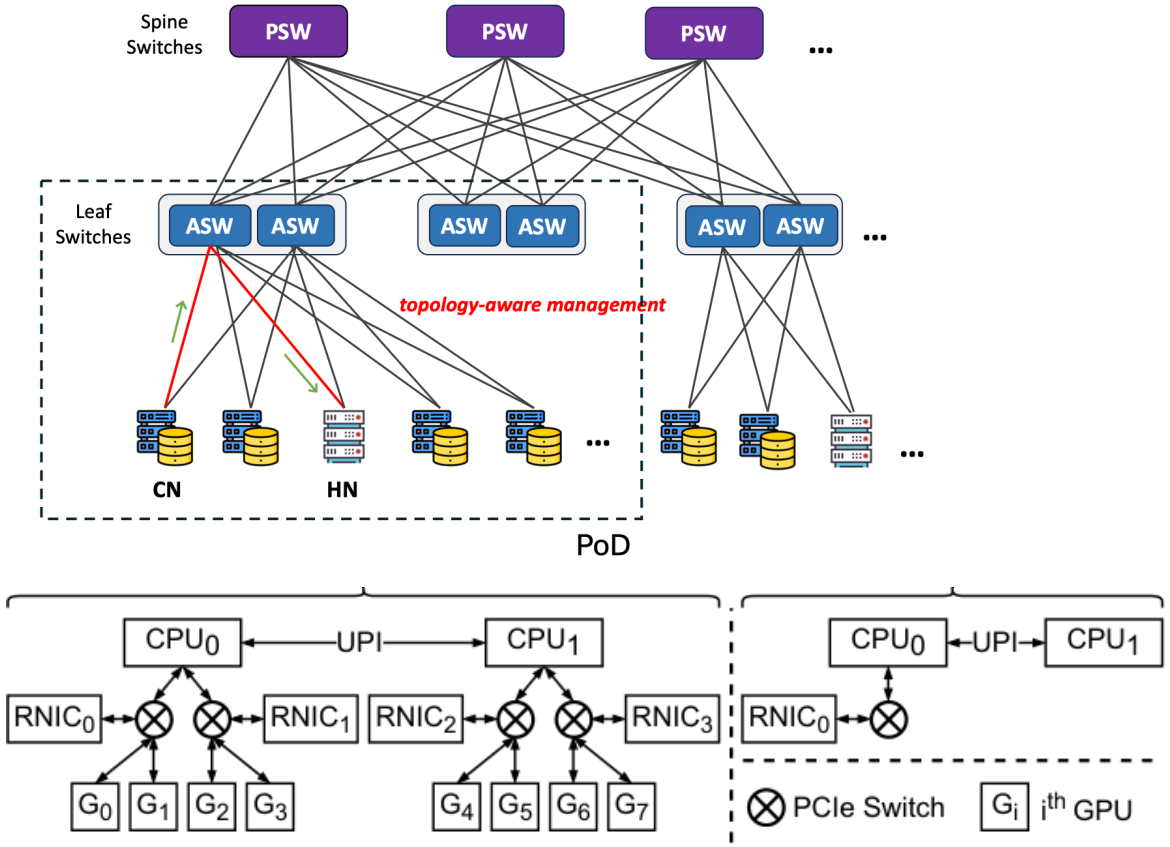


(a) Major CPU operators. (b) Major GPU operators.

Embedding ops take >70% of CPU time; MatMul and attention dominate GPU time (Source: Yang et al., NSDI 2025, Fig. 8)

- Runs **last** in the optimizer pipeline, after JIT fusion, CUDA Graphs, etc., so model owners change nothing
- Seed with GPU-efficient operators, then **DFS-color** upstream and downstream until a CPU-intensive operator is hit
- Transfer-volume optimizations (up to -50%): keep constant subgraphs on the HN, computed once and cached; send a source tensor instead of its larger derived tensors
- 80% of services move **<10 MiB per request**

Key Component 2: Topology-Aware Resource Manager



Inter-node:

- Confine all instances within the same PoD
- Schedule new instance(s) to the ASW with the most existing instances

Intra-node:

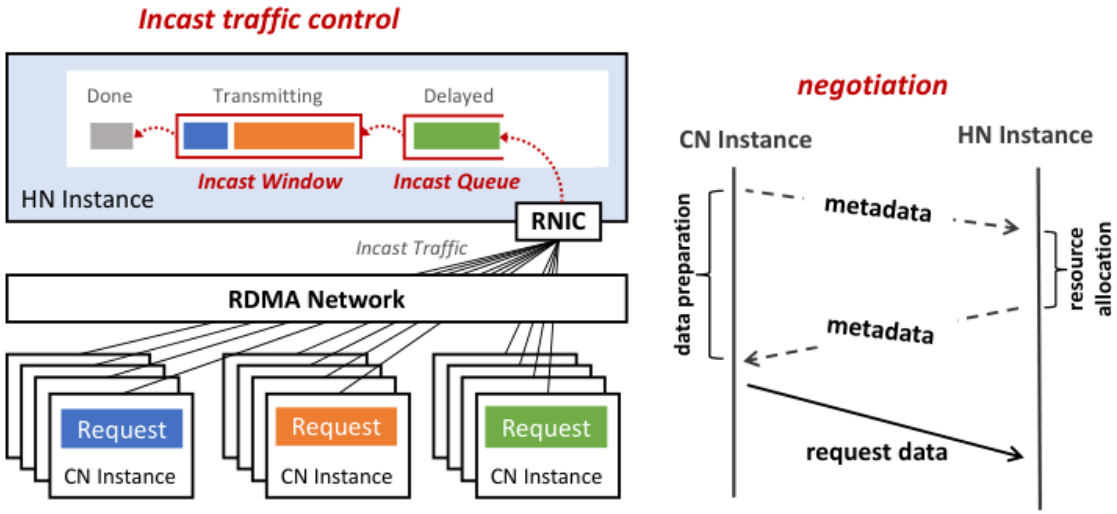
- HN Nodes: bind an instance's GPU and RNIC under the **same PCIe switch**.
- CN Nodes: Prioritize CPU allocation under the same PCIe switch connected to the RNIC

Safety: cap instance density per node and per NIC switch (bandwidth < ~80%).

Key Component 3: SLO-Aware Communication Scheduler

Incast Size	Latency	% of Failed Requests
10	10 ms	≈ 33%
20	40 ms	≈ 50%
100	10 s	≈ 100%

Incast without control: 1,000 requests of 8 MiB (Table 1)

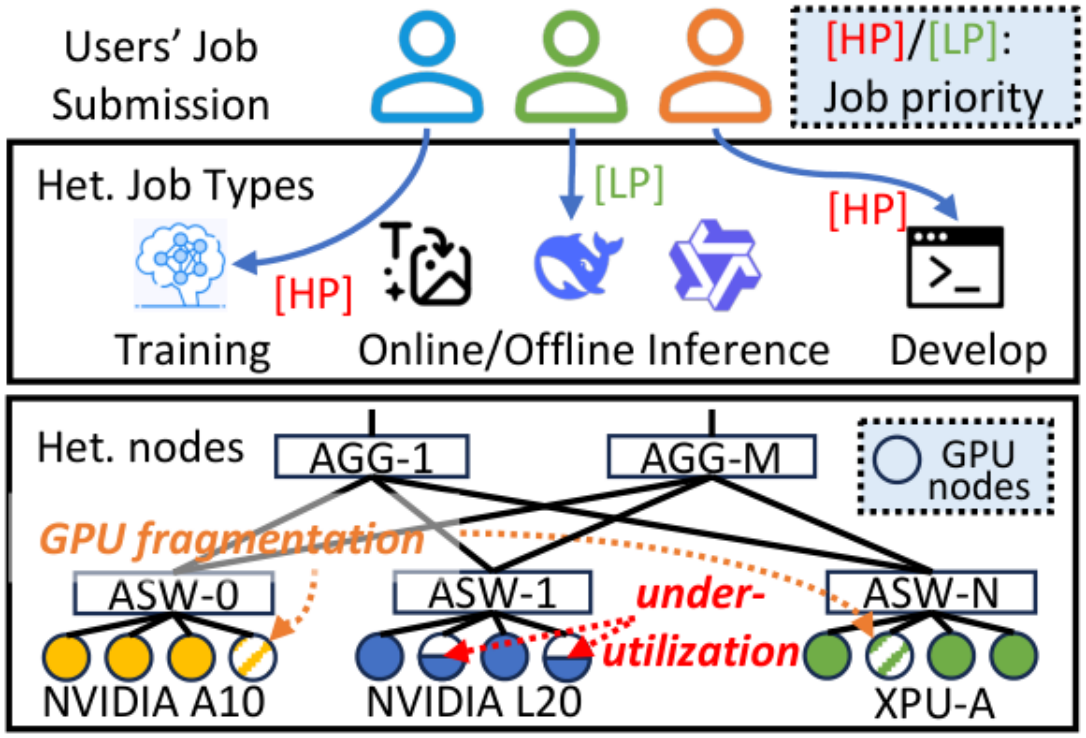


(Source: Yang et al., NSDI 2025, Fig. 12)

- Many CNs fan in to few HNs: **incast**. At 100 senders, latency hits 10 s and nearly all requests fail
- **Incast window:** a CN sends metadata first; the HN admits it only if the window has room, else it waits in the incast queue
- **Adaptive:** RNIC congestion notifications (CNPs) size the window
- **Earliest-deadline-first:** deadline = now + SLO - size/bandwidth, so tight-SLO requests go first in the incast queue

Part II: Heterogeneity at Hyperscale: Characterization and Scheduling of Large Production AI Clusters at Alibaba (Operational Systems)

Alibaba Serverless Infrastructure (ASI)

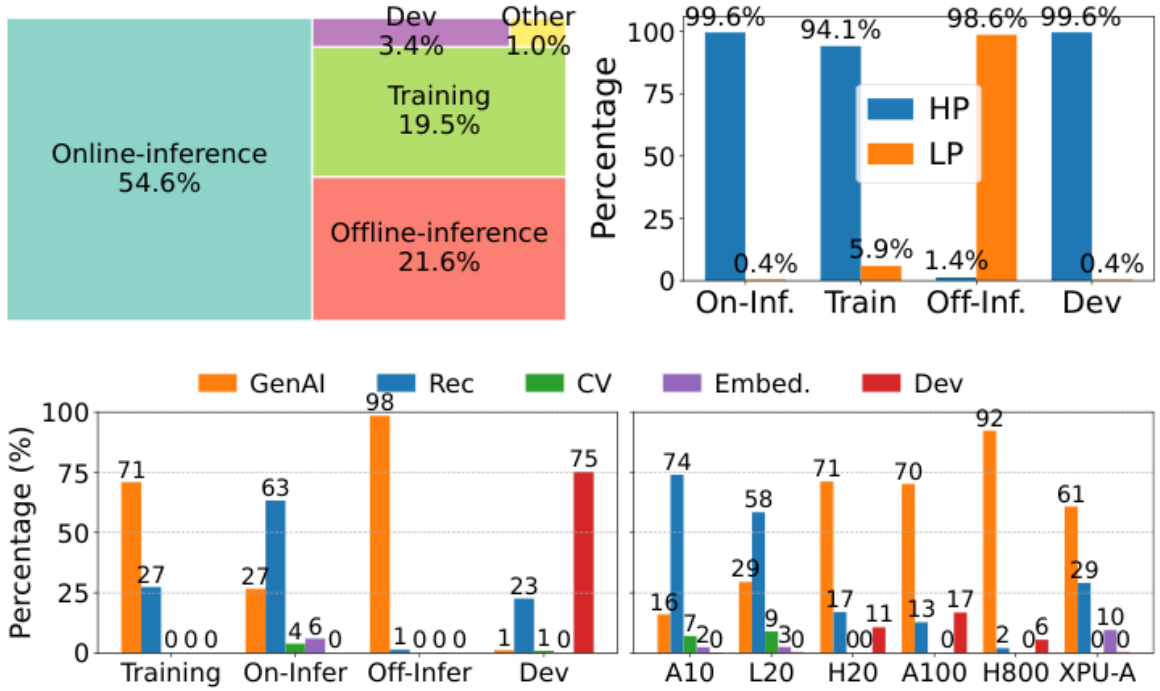


(Source: Li et al., "Heterogeneity at Hyperscale," OSDI 2026, Fig. 1)

Trace	ASI	Acme	PAI	Helios
Year	2025	2023	2020	2020
GPUs	155k	4.7k	6.7k	6.4k

- One shared GPU cluster for **81 departments**: **155,410 GPUs on 37,707 nodes**, 11+ GPU models from several vendors (H20, H800, A100, A10, L20, "XPU-A..E")
- Six-month trace, **14M jobs**: training, development, online and offline inference of LLMs, diffusion, DNNs, Rec
- Fat-tree: 8-GPU nodes under an access switch (ASW, 256-512 GPUs), then aggregation switches
- Priorities: **HP** (guaranteed) and **LP spot** (cheaper, preemptible)

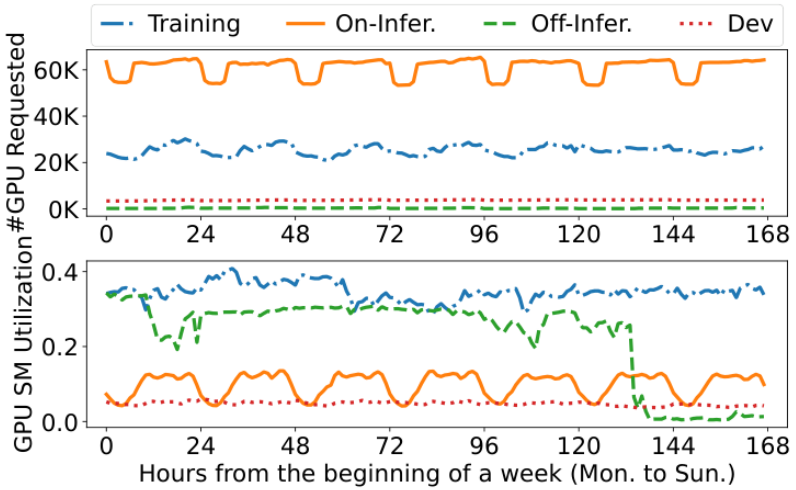
What Runs on ASI?



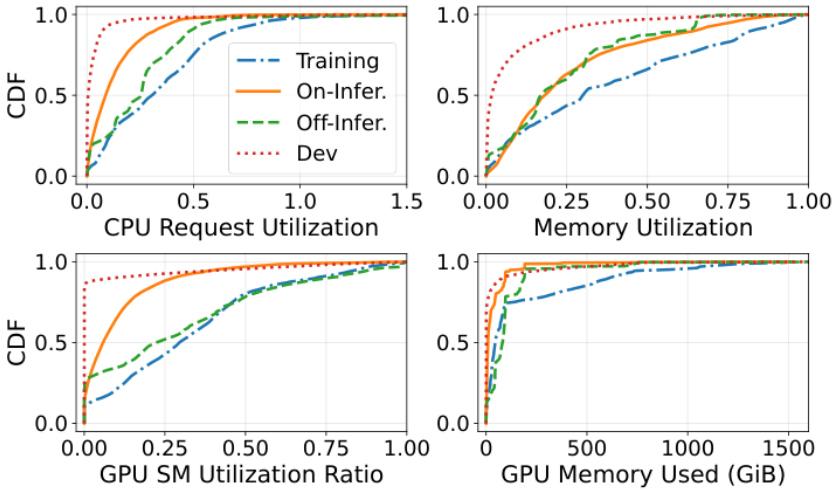
(Source: Li et al., OSDI 2026, Figs. 2-3)

- **Online inference is 54.6%** of GPU hours and almost all HP; offline inference is mostly LP
- GenAI dominates training (71%) and offline inference (98%); online inference is still 63% recommendation
- Specialization by choice: A10/L20 run Rec and CV, H800 is 92% GenAI; **>99% of jobs pin a GPU model**
- Median job: **5 hours** (Acme: 2 min); median GPU request 2 (mean 11); largest GenAI job >2,000 GPUs

Utilization Patterns



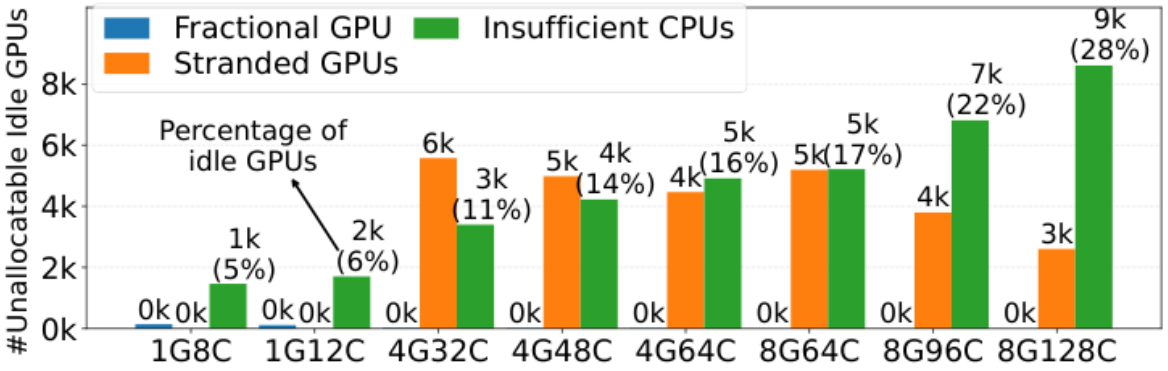
Requested GPUs and SM utilization over a week (Fig. 5)



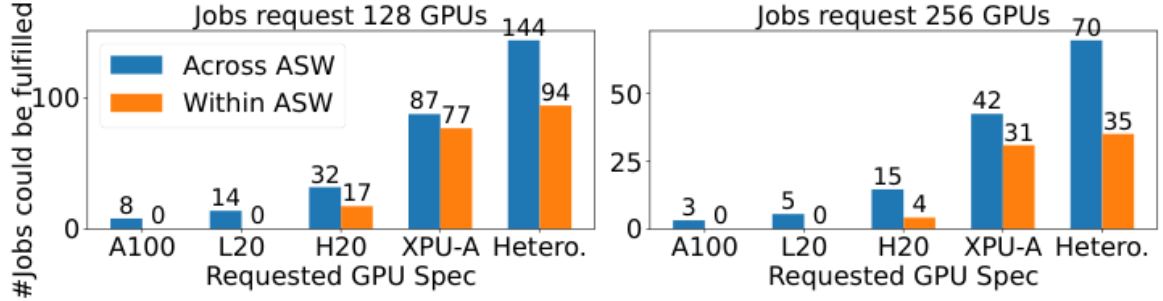
Task-level utilization by job type (Fig. 11)

- Demand is diurnal
- Online inference: **median SM utilization 6%**, memory 30%. GenAI serving fills 94% of memory but 5% of SMs; DNN serving uses 20% memory, 6% SMs

High GPU Demand ≠ High Utilization



Unallocatable idle GPUs by request shape, e.g. 8G128C = 8 GPUs + 128 CPUs (Fig. 13)

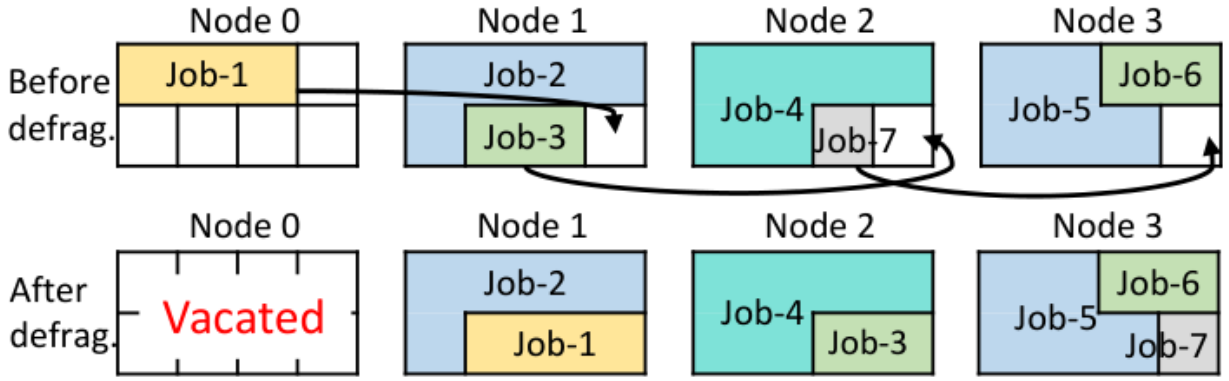


Jobs that fit when all GPUs must share one ASW (Fig. 14)

Four Reasons Idle GPUs Cannot Be Allocated

1. **Headroom:** Production reservation for burst capacity
2. **Stranded GPUs:** Partial node allocation leaves unused GPUs (e.g. a 2-GPU task on an 8-GPU node blocks the next 8-GPU request)
3. **Insufficient CPUs:** CPU/GPU ratio mismatch blocks GPU allocation
4. **Topology**: ASW-locality constraints for large GenAI jobs

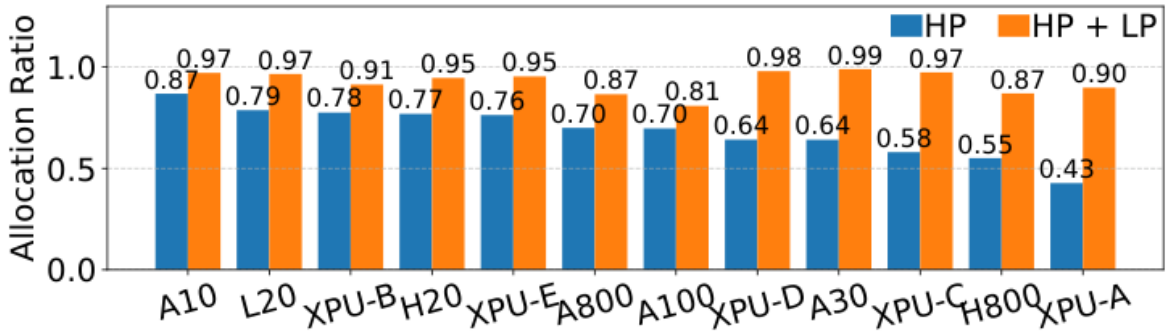
Optimization 1: IPC (Iterative Partitioned Consolidation)



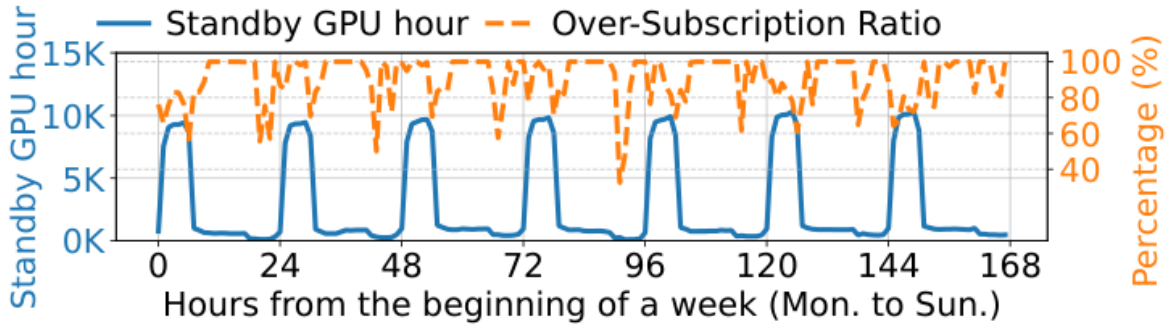
(Source: Li et al., OSDI 2026, Fig. 15: to vacate Node 0, Job-3 moves to Node 2 after Job-7 moves to Node 3, then Job-1 moves to Node 1)

- Goal: Vacate as many nodes as possible by migrating tasks
- Design Choices:
  - **Divide and conquer**: partition cluster into disjoint group, make decision fast
  - **Ejection-chain scheduling**: respect (anti-)affinity and locked-task constraints
  - **Iterate**: drives vacated number higher across rounds
- Decision in **<2 min**; nodes with slack **-20.2%**.
- Companion: topology-aware allocation picks the ASW minimizing the job's GPU **entropy**

Optimization 2: SpotGPU: Harvesting Reserved-but-Idle Capacity



Allocation ratio, HP only vs HP + LP spot (Fig. 9)



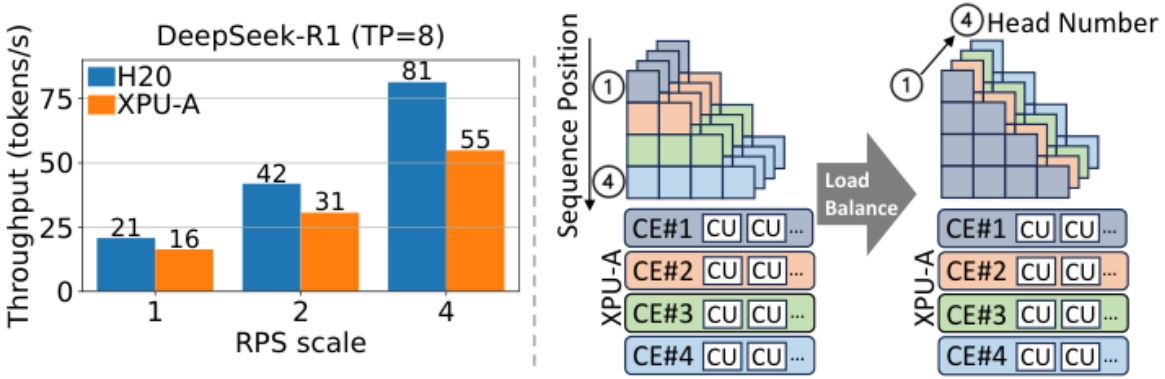
"Standby" GPU hours released by HP tasks (Fig. 16)

- **Standby:** an HP user labels pods releasable; the container stays warm with `sleep` as its main process, so reactivation skips scheduling and image pulls. **10,000 GPU hours** at midnight, 90% harvested
- **Eviction:** SIGTERM, 60 s grace, GPU reclaimed; average 13 s, <5% need SIGKILL
- **Preemptive scheduling**: minimize wasted computation (preemption cost = GPUs × time since last checkpoint)
- Allocation **68% -> 93%**; LP completion -24%

Optimization 3: Heterogeneous GPUs

- XPU-A beats H20 on paper, but out-of-box Gen-AI performance is only 80% due to kernel mismatches with HW features
- Key Optimizations:
  - **Prefill:** FlashAttention kernel leaves one compute engine with most of the work; re-dividing it by attention head balanced the load across engines and sped up prefill by 1.58x.

	#GPUs per node	Memory per card	HBM BW (TBps)	Inter–GPU BW (GBps)	FP16 Tensor Core Compute Spec.
H20	8	96	4.0	900	148 TFLOPS
XPU-A	8	192	5.2	896	203 TFLOPS



(Source: Li et al., OSDI 2026, Table 2, Fig. 17)

- **Decode:** When PagedAttention was compiled for XPU-A, the Triton compiler mis-laid the thread blocks so each thread group redundantly recomputed the whole KV-cache lookup; custom compiler passes restored the intended parallel split.